Quadcopter Control

Helmers, Ole

June 2025

1 What is the point of this project?

I was bored. I wanted to test my modeling and simulation knowledge, but now it also includes nonlinear MPC control. This was mostly done over the weekend, so don't expect much detail.

2 System

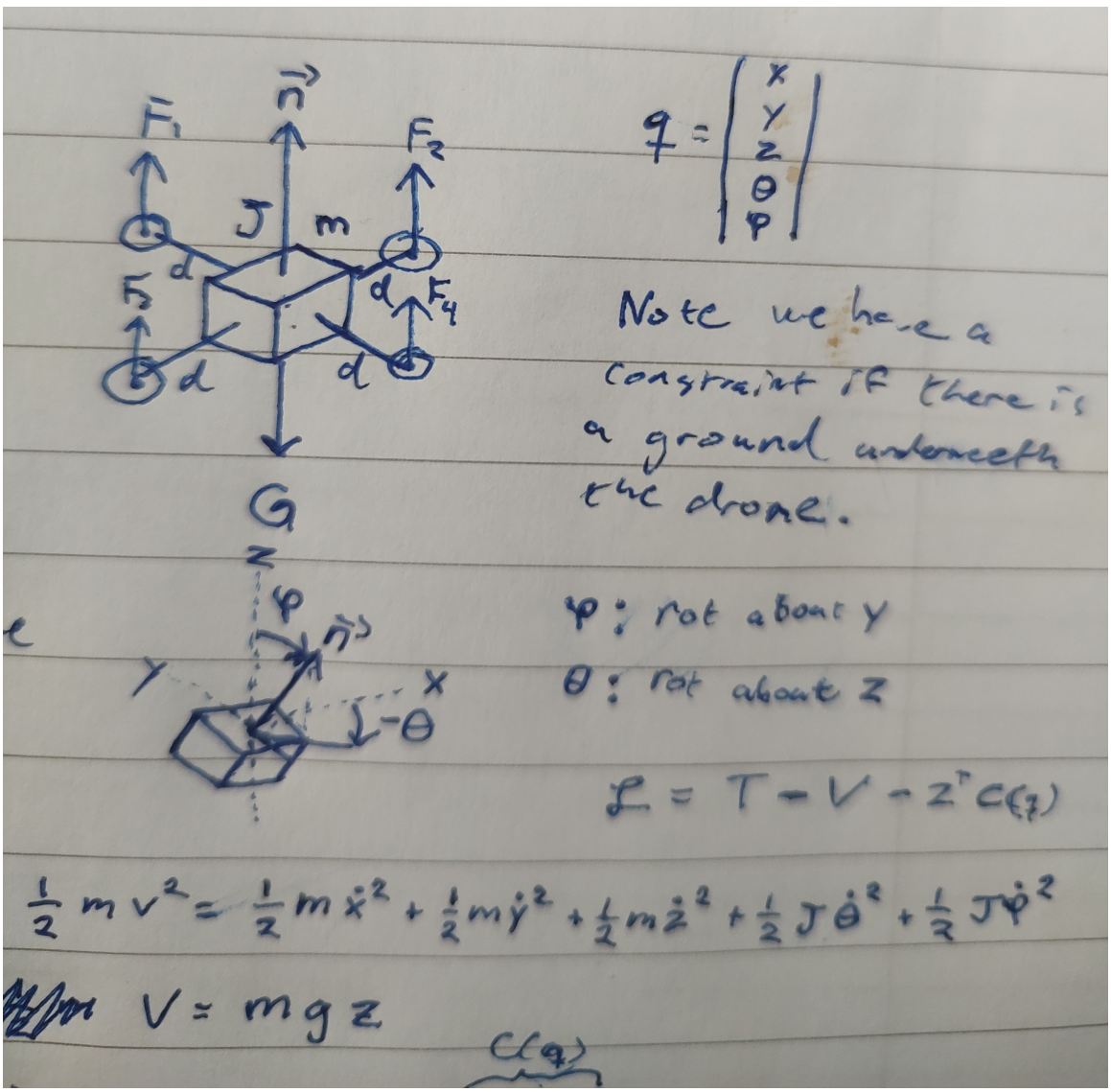

Figure 1: Hand drawing of the system with the yaw state missing. i did not want to re draw it.

The generalized coordinates:

$$q = \begin{vmatrix} x \\ y \\ z \\ \phi \\ \theta \\ \psi \end{vmatrix} \tag{1}$$

From the top we have the 3d position, then roll pitch and yaw.

We have 4 propeller arms in a + configuration, not an x. With

- Motor 1 F_1 (front): $\mathbf{r}_1 = (d, 0, 0)$
- Motor 2 F_2 (right): $\mathbf{r}_2 = (0, d, 0)$
- Motor 3 F_3 (back): $\mathbf{r}_3 = (-d, 0, 0)$
- Motor 4 F_4 (left): $\mathbf{r}_4 = (0, -d, 0)$

3 Derivation

We want to solve:

$$\frac{d}{dt}\left(\frac{\partial \mathcal{L}}{\partial \dot{q}}\right) - \frac{\partial \mathcal{L}}{\partial q} = Q \tag{2}$$

Lagrangian is

$$\mathcal{L} = T - V - c(q) \tag{3}$$

T and V is the kinetic and potential energy respectively.

We mathematically would use constrained lagrange to include the ground by defining

$$c(q) = z - \lambda, \quad \lambda \geq 0 \tag{4}$$

The generalized forces:

$$Q_j = \sum_{i=1}^{n} F_i \frac{\partial r_i}{\partial q_j} \tag{5}$$

Now we start the derivation:

$$T = \frac{1}{2}\dot{q}^T \begin{vmatrix} m & 0 & 0 & 0 & 0 & 0 \\ 0 & m & 0 & 0 & 0 & 0 \\ 0 & 0 & m & 0 & 0 & 0 \\ 0 & 0 & 0 & J & 0 & 0 \\ 0 & 0 & 0 & 0 & J & 0 \\ 0 & 0 & 0 & 0 & 0 & J \end{vmatrix} \dot{q} \tag{6}$$

$$V = mg \begin{vmatrix} 0 & 0 & 1 & 0 & 0 & 0 \end{vmatrix} q \tag{7}$$

This all gives:

$$m\ddot{x} = Q_x \tag{8}$$

$$m\ddot{y} = Q_y \tag{9}$$

$$m\ddot{z} = Q_z - mg \tag{10}$$

$$J\ddot{\phi} = Q_\phi \tag{11}$$

$$J\ddot{\theta} = Q_\theta \tag{12}$$

$$J\ddot{\psi} = Q_\psi \tag{13}$$

Now to the generalized forces. The forces in the body frame:

$$F_{body} = \begin{vmatrix} 0 \\ 0 \\ F_1 + F_2 + F_3 + F_4 \end{vmatrix} \tag{14}$$

so the forces in the inertial frame is:

$$F_{inertial} = R_x(\phi) R_y(\theta) R_z(\psi) F_{body} \tag{15}$$

We use the awful roll pitch and yaw matrix:

$$R_x(\phi)R_y(\theta)R_z(\psi) = \begin{vmatrix} cos(\psi)cos(\theta) & sin(\psi)cos(\theta) & -sin(\theta) \\ cos(\psi)sin(\theta)sin(\phi) - sin(\psi)cos(\phi) & sin(\psi)sin(\theta)sin(\phi) + cos(\psi)cos(\phi) & cos(\theta)sin(\phi) \\ cos(\psi)sin(\theta)cos(\phi) + sin(\psi)sin(\phi) & sin(\psi)sin(\theta)cos(\phi) - cos(\psi)sin(\phi) & cos(\theta)cos(\phi) \end{vmatrix} \tag{16}$$

$$Q_x = \sum_{i=1}^{n} F_i \frac{\partial r_x}{\partial q_j} = (F_1 + F_2 + F_3 + F_4)(cos(\phi)sin(\theta)cos(\psi) + sin(\phi)sin(\psi)) \tag{17}$$

$$Q_y = \sum_{i=1}^{n} F_i \frac{\partial r_y}{\partial q_j} = (F_1 + F_2 + F_3 + F_4)(cos(\phi)sin(\theta)sin(\psi) - sin(\phi)cos(\psi)) \tag{18}$$

$$Q_z = \sum_{i=1}^{n} F_i \frac{\partial r_z}{\partial q_j} = (F_1 + F_2 + F_3 + F_4)(cos(\phi)cos(\theta)) \tag{19}$$

$$Q_\phi = \sum_{i=1}^{n} F_i \frac{\partial r_\phi}{\partial q_j} = d(F_2 - F_4) \tag{20}$$

$$Q_\theta = \sum_{i=1}^{n} F_i \frac{\partial r_\theta}{\partial q_j} = d(F_3 - F_1) \tag{21}$$

$$Q_\psi = \sum_{i=1}^{n} F_i \frac{\partial r_\psi}{\partial q_j} = \kappa(-F_1 + F_2 - F_3 + F_4) \tag{22}$$

κ is the motor torque coefficient or reaction torque constant.

4 Equations

Now we have the equations:

$$m\ddot{x} = (F_1 + F_2 + F_3 + F_4)\big(\cos(\phi)\sin(\theta)\cos(\psi) + \sin(\phi)\sin(\psi)\big) \tag{23}$$

$$m\ddot{y} = (F_1 + F_2 + F_3 + F_4)\big(\cos(\phi)\sin(\theta)\sin(\psi) - \sin(\phi)\cos(\psi)\big) \tag{24}$$

$$m\ddot{z} = (F_1 + F_2 + F_3 + F_4)\cos(\phi)\cos(\theta) - mg \tag{25}$$

$$J\ddot{\phi} = d(F_2 - F_4) \tag{26}$$

$$J\ddot{\theta} = d(F_3 - F_1) \tag{27}$$

$$J\ddot{\psi} = \kappa(-F_1 + F_2 - F_3 + F_4) \tag{28}$$

Remarks

These are second-order differential equations in the generalized coordinates. The code uses rungekutta 4 to simulate this. The constraint $c(q)$ can be enforced with a penalty method or Lagrange multipliers if the drone is to remain above ground.

Nonlinear MPC

Nonlinear MPC is in principle straightforward.

$$\begin{aligned} \min_z \quad f(z) = \sum_{t=0}^{N-1} \left(\frac{1}{2} x_t^T Q_t x_t + d_t^T x_t \right) + \frac{1}{2} x_{t+1}^T Q_{t+1} x_{t+1} \\ + \frac{1}{2} u_t^T R_t u_t + d_t^T u_t + \frac{1}{2} \Delta u_t^T R_{\Delta t} \Delta u_t \end{aligned} \tag{29}$$

subject to

$$x_{t+1} = g(x_t, u_t) \tag{30}$$

$$x_0, u_{-1} = \text{given} \tag{31}$$

$$x_{\text{low}} \leq x_t \leq x_{\text{high}} \tag{32}$$

$$-\Delta u_{\text{high}} \leq \Delta u_t \leq \Delta u_{\text{high}} \tag{33}$$

$$u_{\text{low}} \leq u_t \leq u_{\text{high}} \tag{34}$$

where

$$Q_t \succ 0, \quad R_t \succ 0, \quad R_{\Delta t} \succ 0 \tag{35}$$

The nonlinearities are found in $g(x_t, u_t)$

Implementation Details

We define the state vector x_t and control input u_t as:

$$x_t = \begin{bmatrix} x & y & z & \phi & \theta & \psi & \dot{x} & \dot{y} & \dot{z} & \dot{\phi} & \dot{\theta} & \dot{\psi} \end{bmatrix}^T \tag{36}$$

$$u_t = \begin{bmatrix} F_1 & F_2 & F_3 & F_4 \end{bmatrix}^T \tag{37}$$

The nonlinear dynamics $x_{t+1} = g(x_t, u_t)$ are derived from the second-order equations presented previously. We discretize them using a time step Δt with a 4th-order Runge-Kutta method:

$$x_{t+1} = x_t + \int_t^{t+\Delta t} \dot{x}(\tau)\, d\tau \approx \text{RK4}(x_t, u_t) \tag{38}$$

Cost Function Weights

The cost function penalizes deviations from reference trajectories x_t^{ref} and u_t^{ref}, as well as large control effort and input rate changes:

$$f(z) = \sum_{t=0}^{N-1} \left[\|x_t - x_t^{\text{ref}}\|_Q^2 + \|u_t - u_t^{\text{ref}}\|_R^2 + \|\Delta u_t\|_{R_\Delta}^2 \right] + \|x_N - x_N^{\text{ref}}\|_{Q_N}^2 \tag{39}$$

where Q, R, R_Δ, Q_N are positive-definite weighting matrices.

Solver

The resulting problem is a nonlinear program (NLP) with nonlinear dynamics and constraints. It can be solved using Sequential Quadratic Programming (SQP) or Interior Point methods.

We solve the optimization problem at each control interval in a receding horizon manner:

1. At time t, measure or estimate the state x_t.
2. Solve the NLP to find the optimal sequence $\{u_t, u_{t+1}, \ldots, u_{t+N-1}\}$.
3. Apply u_t to the system.
4. Repeat the procedure at $t + 1$.

4.1 Cost Function and Constraints

To ensure the implementation remains computationally efficient, most state constraints are implemented as penalty functions within the objective function $f(z)$. The total cost is:

$$f(u_{0\ldots N-1}) = \sum_{t=0}^{N-1} \left(L_{tracking} + L_{damping} + L_{stability} + L_{control} + L_{ground} \right) \tag{40}$$

Consistent with the C++ implementation, the components are defined as:

- **Tracking:** $1000 \cdot \|p_t - p_{ref}\|^2$ (Penalizes position error).
- **Damping:** $40 \cdot \|\dot{p}_t\|^2$ (Prevents oscillations).
- **Stability:** $50 \cdot (\phi^2 + \theta^2 + \psi^2)$ (Keeps the drone level).

- **Control Effort:** $0.01 \cdot \sum(F_i - \frac{Mg}{4})^2$ (Penalizes deviation from hover thrust).
- **Ground Penalty:** If $z < \lambda$, we add $5000 \cdot (\lambda - z)^2$ to the cost.

The only **hard constraints** applied are the physical motor limits:

$$0.0 \leq F_i \leq 15.0 \text{ N} \tag{41}$$

4.2 Solver: COBYLA

While many NMPC formulations use SQP, this project utilizes the **LN_COBYLA** algorithm from the **NLopt** library. COBYLA (Constrained Optimization BY Linear Approximations) is a derivative-free optimization algorithm that is robust for nonlinear problems with inequality constraints.

Algorithm 1 NMPC Loop in C++

```
Measure current state s = [p, η, ṗ, η̇]
Define 4N optimization variables for thrust sequence
Set objective to MPCController::cost_function
Set bounds [0, 15] for all thrust variables
Run nlopt::optimize
Apply F_{0...3} to the Rigid Body Simulator
```

5 Simulation Environment

The simulation is written in C++ and consists of two main threads:

1. **Physics Thread:** Integrates the nonlinear equations of motion using a 4th-order Runge-Kutta (RK4) method at a fixed $\Delta t = 0.01s$.
2. **Visualization Thread:** Uses OpenGL (GLFW/GLEW) to render the quadcopter's state in real-time.

6 Conclusion

The NMPC successfully stabilizes the quadcopter and follows 3D position references. By utilizing penalty functions for ground avoidance and stability, the optimization remains feasible even when the system approaches physical boundaries. Future work could involve replacing COBYLA with a gradient-based solver like SLSQP to reduce the computational cost per iteration.